

项目文档：架构优化与功能实现

一、开发者说明

- 1. 成员1:**
学号：12412314
姓名：金艺涵
负责工作：matrix_calc_top模块、matrix_input模块、matrix_settings模块，还有卷积相关的PPT制作、录制，以及项目文档的撰写
贡献百分比：33%
- 2. 成员2:**
学号：12412301
姓名：杨伟铭
负责工作：matrix_storage模块、matrix_generate模块、uart_rx.v模块、uart_tx.v模块，参与卷积PPT制作、录制和项目文档撰写
贡献百分比：33%
- 3. 成员3:**
学号：12412307
姓名：范晓乐
负责工作：matrix_operation模块、matrix_display模块，以及卷积PPT制作、录制和项目文档撰写
贡献百分比：33%

二、开发计划日程安排和实施情况、版本修改记录

1. 开发计划与实施情况

阶段	计划时间节点	计划任务	实际完成时间	实际完成情况 (是否按时/调整说明)
需求分析	2025.11.1-2025.11.20	确定项目功能边界、拆分模块，确定分工	2025.11.20	按时完成，补充了bonus功能的需求，为未来做了规划
架构设计	2025.11.21-2025.12.1	输出初始架构文档、确定模块接口，brain	2025.12.1	按时完成，接口定义比计划更细化，并分

		storm		配到每一位组员具体的工作和ddl
编码开发	2025.12.2-2025.12.10	各模块代码编写、初步联调	2025.12.11	延迟1天，因各个模块的协同配合存在bug，并且状态机过于冗杂需要手动调试
测试优化	2025.12.12-2025.12.24	功能测试、bug修复、架构优化	2025.12.24	按时完成，优化了模块间耦合度，完成所有功能的测试
文档整理	2025.12.25-2025.12.27	填写项目文档、整理开源/AI记录	2025.12.27	按时完成，顺利结束project

2. GitHub仓库信息

仓库链接：https://github.com/CS207DigitalLogic/final-project-always_no_bug.git

提交记录说明：

1. 2025-12-22: 对于新出现的bug进行了修复 (Samuel)
2. 2025-12-21: 增添了我负责的display和operation模块的详细中文批注 (Justin Fan 7777777)
3. 2025-12-19: 完美地实现了输入错误进行倒计时，并且重新输入正确以后可以结束倒计时这个功能，我简直就是天才 (Samuel)
4. 2025-12-19: 完成了矩阵的标量乘法功能，并且把输出的补齐功能实现了，其它的所有功能都完全正常了，现在只差类型不匹配时的倒计时了 (Samuel)
5. 2025-12-14: 更新了top模块，微调了display，实现多个模式的自动调用 (sparklejin)
6. 2025-12-12: 将display模块彻底推翻重写，使得它与各模块衔接正常 (Justin Fan 7777777)
7. 2025-12-12: 实现了第五版的top模块，对于所有功能的串联做了改进 (Samuel)
8. 2025-12-11: 再次修改了数码管的控制逻辑 (sparklejin)
9. 2025-12-11: 修改了top模块的数码管显示 (sparklejin)
10. 2025-12-08: 实现了完整的计算模块（包括卷积功能），并且完成了设置模块，能自由切换，且实现了第一版的有所有功能的top模块 (Samuel)
11. 2025-12-07: 增加了按键防抖功能，新增key_filter模块并且修改了top模块 (Samuel)
12. 2025-12-06: display模块针对负数问题的解决 (Justin Fan 7777777)
13. 2025-12-06: 完成了generate模块 (sparklejin)
14. 2025-12-06: display模块和uart_tx.v模块 (Justin Fan 7777777)

- 15. 2025-12-06: 完成了第二版的storage(应该是终版) (Samuel)
- 16. 2025-12-06: 先照搬了lab课的uart模块, 并写了初步的input模块 (sparklejin)
- 17. 2025-12-06: 实现了第一版的串口UI以及第一版的storage功能 (Samuel)

三、项目架构设计说明

1. 交互稳定性与用户体验改进 (Interaction & Stability)

- 1. 原架构问题:** 最开始设计的时候, 我们直接用按键 (S0、P15) 做输入, 没考虑去抖动。实际装到硬件上才发现, 机械按键会抖, 按一次可能被识别成好几次, 导致状态机乱跳。而且错误提示就只亮个LED7, 用户根本不知道是啥错、啥时候能恢复。
- 2. 改进点:** 我们加了个key_filter消抖模块, 在matrix_calc_top里实例化, 专门处理BTN_start这些关键输入, 保证信号稳定。另外还加了matrix_settings模块和倒计时逻辑——要是操作不合法 (比如矩阵维度不匹配), 系统就进入倒计时, 七段数码管显示还剩多久解锁, 这样用户体验好多了。
- 3. 最终实现:** key_filter #(.CLK_FREQ(CLK_FREQ)) u_btn (...) 保证输入稳定。错误发生时, 数码管接管显示, 倒计时结束后自动恢复 IDLE 状态。

2. 模块职责解耦 (Module Decoupling & SRP)

- 1. 原架构问题:** 最开始的文档里, matrix_operation模块既要算数据, 还要负责通过UART_tx输出结果, 不符合单一职责原则 (SRP)。这样一来, 计算和显示逻辑缠在一起, 还容易抢UART的发送资源, 出冲突。
- 2. 改进点:** 我们把计算和显示拆开了! 现在matrix_operation只干计算的活, 算完后把结果数据 (res_data)、维度 (res_row、res_col) 和存储使能 (res_store_en) 传给顶层就行。matrix_display则改成了通用的显示引擎, 不光能显示存储里的矩阵, 还能直接接计算模块的输出打印, 甚至加了“统计模式” (显示不同维度矩阵的数量) 和“列表模式”。
- 3. 最终实现:** matrix_operation 纯粹化: 输入操作数 -> 组合逻辑/时序计算 -> 输出结果数据。matrix_display 增强: 增加了mode_select 接口 (00:数据, 01:统计, 10:列表), 并内置了BCD码转换和负号处理逻辑 (S_SEND_SIGN, S_CONVERT_BCD), 支持更复杂的格式化输出。

3. 状态机架构优化 (FSM Optimization)

- 1. 原架构问题:** 最开始想在Top模块里搞一个超大的状态机, 把S_IDLE、S_MENU、S_INPUT、S_CALC这些十几个状态全塞进去, 结果导致顶层逻辑特别臃肿, 后期改起来、维护起来都巨麻烦。
- 2. 改进点:** 最后我们用了分布式状态机, 分层控制。Top模块主要靠SW[2:0]做多路选择器, 选不同的子模块 (输入、生成、显示、计算) 工作就行。具体的业务流程, 比如UART接收数据、计算流水线、打印字符这些细节, 都放到各个子模块自己的状态机里处理, 子模块自己管自己的逻辑。
- 3. 最终实现:** Top 模块代码主要由MUX逻辑组成 (如 storage_wr_en 的选择逻辑), 不再包含复杂的跳转逻辑。matrix_display 内部维护了复杂的打印状态机 (30+个状态), 处理

字符流的细节。

4. 功能扩展与算法增强 (Feature Expansion)

- 1. 原架构问题：**一开始只规划了矩阵加法、乘法、转置这三个基础功能，没考虑负数怎么显示，也没设计复杂的输出格式，后来用的时候才发现这些都得补。
- 2. 改进点：**我们给matrix_operation模块加了高级功能——标量乘法和卷积运算（Bonus功能）。卷积还专门做了硬件加速，设计了流水线输出接口（conv_out_valid、conv_out_val），支持流式输出。显示模块也优化了，能处理有符号数（signed），负数能正确显示负号，输出的矩阵还能自动对齐或换行。
- 3. 最终实现：**支持 5 种操作类型：Transpose, Add, Scalar Mul, Matrix Mul, Convolution。支持参数配置：通过 matrix_settings 动态调整倒计时时间和生成数量。

四、bonus实现说明

本次项目实施两个核心Bonus功能，分别为输出对齐与格式化、卷积运算硬件加速，具体实现说明如下：

1. 输出对齐与格式化 (Output Alignment & Formatting)

设计目标：为了使矩阵数据在串口终端（如 PuTTY, Tera Term）上以直观的二维网格形式显示，而不是杂乱的一维数据流，实现智能的输出对齐与格式化逻辑。

实现细节：该功能主要集成在 matrix_display.v 模块的状态机中。

网格化打印逻辑：

- 状态机维护了行计数器 (r_cnt) 和列计数器 (c_cnt)。
- 列间分隔：在每行元素打印完毕后 (S_DATA_DELIM状态)，系统会自动发送一个 空格 (ASCII 0x20) 作为分隔符，确保同一行的元素之间有清晰的间距。
- 行末换行：当检测到当前列 (c_cnt) 为该行的最后一个元素时，状态机跳转至 S_DATA_LF 状态，发送 回车换行符 (CR+LF, 0x0D 0x0A)。这强制光标移动到下一行的开头，从而在终端上构建出完美的矩阵形状。

数值格式化：

- BCD 转换：模块内置了二进制到 BCD 码 (Binary-Coded Decimal) 的转换逻辑 (S_CONVERT_BCD)，将 FPGA 内部的 16 位二进制补码转换为人类可读的十进制数字。
- 符号处理：支持有符号数显示。在发送数字前，系统会检查符号位 (is_negative)，如果是负数，则先发送 负号 '-' (0x2D) (S_SEND_SIGN)，实现了对负数矩阵元素的完美支持。

代码对应：matrix_display.v: S_DATA_DELIM, S_DATA_LF, S_CONVERT_BCD, S_SEND_SIGN 等状态。

2. 卷积运算硬件加速 (Convolution Hardware Acceleration)

设计目标：作为项目的高级 Bonus 功能，实现基于硬件的卷积加速器，用于图像处理演示。为保证演示的稳定性和可重复性，采用“内置图库 + 滑动窗口”的方案。

实现细节：该功能集成在 `matrix_operation.v` 模块中，操作码为 `op_type = 3'd4`。

内置 ROM 图像源：

1. 为避免手动输入大量像素数据的繁琐，在模块内部实例化了一个 10x12 的 ROM (Read-Only Memory)，预存了一张固定的灰度测试图像 (`rom_image`)。
2. 该图像包含 120 个像素点，数据范围 0-9，用于模拟真实的输入特征图。

滑动窗口流水线 (Sliding Window Pipeline)：

1. 3x3 卷积核：用户输入的 3x3 矩阵 (Matrix A) 作为卷积核 (Kernel/Filter)。
2. 自动遍历：状态机利用 `r_idx` (0-7) 和 `c_idx` (0-9) 自动控制卷积核在 10x12 的图像上滑动，生成 8x10 的输出特征图。

流式输出接口 (Streaming Output)：

1. 不同于普通矩阵运算等待全部完成后才输出，卷积模块采用了流式输出设计。
2. 每计算完一个像素点（即 `k_idx` 遍历完 9 次），立即产生 `conv_out_valid` 脉冲，并输出当前的坐标 (`conv_out_r`, `conv_out_c`) 和像素值 (`conv_out_val`)。这种设计极大地降低了对片上存储资源的需求，并提高了系统的响应速度。

代码对应：`matrix_operation.v`: `op_latched == 3'd4` 分支, `rom_image` 初始化块, `S_CALC` 状态下的 `k_idx` 循环逻辑。

乘累加运算 (MAC)：内部维护了一个 `k_idx` (0-8) 计数器，用来遍历 3x3 窗口里的 9 个像素。每个时钟周期做一次“核权重×图像像素”的乘法，然后把结果累加到 `acc` 寄存器里。

五、开源、AI使用及思考总结

1. 使用网络代码情况说明

我们开发过程中参考了一些开源代码，主要用在基础接口协议的实现上，具体情况如下：

1. UART 串口通信模块

代码来源：GitHub 仓库 `alexforencich/verilog-uart`（或者其他通用 UART 实现，也可能是课程实验课给的基础代码）。

整合方式：我们只用了它底层的字节收发逻辑 (`uart_rx.v`、`uart_tx.v`)，上层的协议解析状态机是我们自己写的 (`matrix_input.v` 和 `matrix_display.v`)。原代码只负责字节级的物理层传输，我们自己加了 ASCII 码解析、多位数字组装、负数处理，还有矩阵数据流的打包和解包逻辑，把“字符流”转换成了能直接用的“矩阵对象”。

2. 按键消抖模块 (Key Filter)

代码来源：常见的 Verilog 教学资源或者技术博客（比如 CSDN、StackOverflow 上的标准消抖例子）。

整合方式：我们直接把这个模块拿来用，处理 `BTN_start` 和 `rst_n` 信号。因为我们项目时钟是 100MHz，还特意调整了模块内部的计数器参数，把消抖时间控制在 10ms-20ms，这样就能避免按键抖动导致状态机误触发了。

2. AI 使用情况及效果

这次项目里，AI助手就像我们的“结对编程伙伴”，从开始到结束都参与了开发。

使用工具：GitHub Copilot, ChatGPT-4o / Claude 3.5 Sonnet (用于逻辑推理和文档生成)。

AI主要用在这些场景：

1. 代码生成与补全：写一些重复率高的Verilog代码，比如状态机的case分支、模块端口定义、参数化模板这些，AI能快速生成，省不少时间。
2. Bug调试与分析：仿真波形出问题的时候，我们把代码片段和异常现象发给AI，它能帮我们快速定位逻辑漏洞，比如matrix_operation模块里乘法累加器位宽溢出的问题，就是AI提示我们的。
3. 文档撰写辅助：帮我们写模块接口说明、状态机描述，还有现在这份总结报告的初稿，我们再根据实际情况修改。
4. 架构设计建议：项目刚启动的时候，我们问了AI关于模块划分的建议，最后确定了“计算和显示分开”的架构思路，少走了很多弯路。

AI辅助开发的好处：

1. 效率翻倍：不用自己写大量样板代码，省下来的时间能专心研究核心算法逻辑。
2. 快速查知识：遇到Verilog语法细节，比如signed运算规则、function怎么用，AI给的示例比查手册更直观，一看就懂。
3. 补全思维漏洞：AI经常能提醒我们一些容易忽略的边界情况，比如输入为空、矩阵维度不匹配、负数显示异常这些，帮我们提前规避了不少问题。

AI辅助开发的坑：

1. 会瞎编逻辑：AI有时候写的代码语法没问题，但逻辑是错的，特别是时序逻辑！它会在时序逻辑里混用阻塞赋值，一仿真就出问题。
2. 全局理解差：AI没法完全搞懂整个项目的时序约束，写的局部代码可能会破坏整体时序，还得我们自己调整。
3. 容易依赖：要是不加思考直接用AI的代码，很可能搞不懂底层逻辑，后期维护的时候就麻烦了。

3. 提示词工程展示

下面是三个我们实际开发中用的提示词例子，能看出来我们和AI“协作迭代”的过程：

实例一：矩阵乘法逻辑实现

1. 输入提示词：“请帮我写一个 Verilog 模块，实现两个 5x5 矩阵的乘法。要求使用状态机，支持 16 位有符号数。”
2. AI 回复内容：(生成了一个简单的三层for 循环嵌套在 always 块中的代码，试图在一个时钟周期内完成所有计算)。
3. 我们的调整思路：发现AI写的代码在硬件上要么用不了（不可综合），要么时序特别差——硬件跟软件不一样，没法在一个时钟周期里跑完三层循环。所以我们重新写了提示词：“请重写矩阵乘法逻辑。不要使用 for 循环。请设计一个状态机，利用计数器 i, j, k，每个时钟周

期只计算一次乘法累加 (MAC)，并将结果存入累加器。当计算完一个元素后，再写入结果 RAM。”-> 最后终于得到了能用上的流水线式matrix_operation.v。

实例二：UART 接收状态机

1. 输入提示词：“写一个状态机解析 UART 输入的矩阵数据，格式是 1 2 3 4 ...，用空格分隔。”
2. AI 回复内容：(生成了一个能解析正整数的状态机，但忽略了负数和多余空格的处理)。
3. 我们的调整思路：实际用的时候可能会输入负数（比如-5），用户也可能不小心输多个空格，代码得扛住这些情况才行。所以我们改了提示词：“修改状态机：1. 增加对 负号 '-' 的检测和处理（补码转换）。2. 增加对 连续空格 的容错处理，即忽略多余的空格，只在检测到数字后才开始解析。”-> 这样matrix_input.v就更稳定了。

实例三：卷积功能实现

1. 输入提示词：“我想在项目中加一个 Bonus 功能，做卷积运算。怎么实现比较好？”
2. AI 回复内容：“你可以让用户输入一个大矩阵和一个卷积核，然后计算。”(方案平平无奇，且输入太麻烦)。
3. 我们的调整思路：答辩时间有限，让用户输入大矩阵太费时间了，得搞个能快速演示效果的方案。所以改进提示词：“请设计一个方案：在Verilog内部硬编码 (Hardcode)一个10x12的ROM图像。用户只需输入3x3的卷积核。模块自动扫描ROM图像，并流式输出计算结果。请给出ROM初始化和滑动窗口计算的代码片段。”-> 这就有了现在的Bonus卷积演示方案。

4. 个人反思和总结

1. 金艺涵：我主要扛了Top顶层模块的设计，还要协调三个子模块的衔接，现在想起来都觉得头大！最开始没做好模块化，把一堆逻辑堆在一起，后续联调的时候直接崩了——一个小bug能熬大夜找半天，模块之间信号串扰、时序对不上，当时真的有点想放弃。后来我们咬牙重构，把显示、计算、存储拆得明明白白，还给每个模块定了统一的握手信号，比如start、done这些，调试起来才终于顺畅了。当最后整个系统稳定跑通的时候，那种成就感真的没法说！另外我还学会了用AI生成Testbench，比自己手动写快多了，省下来的时间能专心抠核心逻辑，这波血赚。

2. 杨伟铭：我负责的是matrix_display和matrix_input这两个“门面模块”，看起来是和用户交互，实际全是细节坑！为了让串口输出的矩阵对齐工整，还有负数能正确显示，我跟ASCII码、状态机死磕了好久。经常出现负号显示错位、列与列对不齐的情况，有时候改完一个问题又冒出新问题，调试到怀疑人生。比如最开始没考虑连续空格的情况，用户多输一个空格，整个解析就乱了，只能重新梳理状态机逻辑。但当最后看到终端上整整齐齐的矩阵、负数也能正确显示的时候，真的超兴奋！现在我对UART协议和FSM的理解，再也不是书本上的理论了，都是实打实调出来的经验。

3. 范晓乐：核心的matrix_operation模块和Bonus卷积功能由我来负责，其中卷积硬件加速真的快把我逼疯了！一开始我总带着软件写循环的思路来设计，写出来的代码根本不可综合，时序也完全不达标，卡了整整三天没进展。后来只能推翻重写，慢慢从“软件思维”转到“硬件思维”，学着用流水线的方式设计，还用ROM预存图像省时间，复用乘法器省资源。当看到卷积核

在预设图像上滑动，流式输出的结果全对的时候，我当场就在实验室喊出来了！那种突破思维瓶颈、搞定硬骨头的感觉太爽了！AI在这过程中帮我理解了卷积的硬件映射逻辑，不然我可能还要多熬几天。

5. 项目和课程改进建议

针对 AI 大模型时代的数字逻辑课程创新：

1. 从“手写代码”转向“架构设计与验证”：现在AI写基础Verilog代码超快，我们觉得课程考核真的不用再死磕“会不会写语法正确的代码”了！反而应该重点放在“能不能设计合理的硬件架构”和“会不会验AI写的代码”上。我们项目里就吃了亏，AI生成的代码语法没问题，但时序巨差，后来花了好多时间优化。建议加个Code Review环节，让大家上台讲自己怎么挑AI代码的错，比如锁存器、时序违例这些坑，这样比单纯写代码记得牢多了。

2. 引入“AI 结对编程”实验：可以借鉴斯坦福那个vibe-coding课程，专门整个实验让我们用AI做复杂模块，比如FIFO或者SPI控制器。要求提交提示词记录、AI写的错代码，还有我们改对后的代码。这样既能练怎么跟AI高效沟通（写提示词），还能培养批判性思维——毕竟AI写的不是都对，我们项目里就踩过AI的坑，不仔细检查真的会出大问题。

3. 项目选题的开放性与系统性：结课项目可以多搞点软硬结合的题目！比如我们这次用Python通过串口跟FPGA交互，就比单纯做波形仿真有意思多了，还能体验从算法设计到最终跑通硬件的全流程。建议少局限在纯仿真，多强调板级调试和系统集成，我们调试硬件的时候虽然熬了好多夜，但最后看到实物跑通的那一刻，获得感比任何时候都强！

4. 增加“时序约束”与“资源优化”的权重：AI写代码根本不考虑时序和资源，这也是我们项目后期最头疼的问题之一——代码能跑但时序不收敛，只能一点点优化关键路径、复用DSP资源。这些高阶技能AI现在还替代不了，课程应该多重点讲这些，早学早受益，能少走很多弯路，也是未来做硬件工程师的核心本事。